

Software Requirements: Analysis and Specification

Analysis of Gathered Requirements (CONT.)

- Requirements analysis involves:
 - Obtaining a clear, in-depth understanding of the product to be developed,
 - Remove all ambiguities and inconsistencies from the initial customer perception of the problem.
- It is quite difficult to obtain:
 - A clear, in-depth understanding of the problem:
 - Especially if there is no working model of the problem.
- Experienced analysts take considerable time:
 - To understand the exact requirements the customer has in his mind.

Analysis of Gathered Requirements (CONT.)

- Experienced systems analysts know - often as a result of painful experiences
 - Without a clear understanding of the problem, it is impossible to develop a satisfactory system.
- Several things about the project should be clearly understood by the analyst:
 - What is the problem?
 - Why is it important to solve the problem?
 - What are the possible solutions to the problem?
 - What complexities might arise while solving the problem?

Analysis of Gathered Requirements (CONT.)

- Some anomalies and inconsistencies can be very subtle:
 - Escape even most experienced eyes.
 - If a formal model of the system is constructed,
 - Many of the subtle anomalies and inconsistencies get detected.
- After collecting all data regarding the system to be developed,
 - Remove all inconsistencies and anomalies from the requirements,
 - Systematically organize requirements into a Software Requirements Specification (SRS) document.

Software Requirements Specification

- Main aim of requirements specification:
 - Systematically organize the requirements arrived during requirements analysis.
 - Document requirements properly.
- SRS document is useful in various contexts:
 - Statement of user needs
 - Contract document
 - Reference document
 - Definition for implementation

Software Requirements Specification: A Contract Document

- Requirements document is a reference document.
- **SRS document is a contract between the development team and the customer.**
 - Once SRS document is approved by customer,
 - Any subsequent controversies are settled by referring the SRS document.

Software Requirements Specification: A Contract Document

- Once customer agrees to SRS document:
 - Development team starts to develop (rather prepare **a detailed design**) the product according to the requirements recorded in the SRS document.
- The final product will be acceptable to the customer:
 - **As long as it satisfies all the requirements recorded in the SRS document.**

SRS Document (CONT.)

- SRS document concentrates on:
 - What needs to be done
 - Carefully avoids the solution ("how to do") aspects.
- The SRS document serves as a contract
 - Between development team and the customer.
 - Should be carefully written
- The requirements at this stage:
 - Written using end-user terminology.
- If necessary:
 - Later a formal requirement specification may be developed from it.

Properties of a Good SRS Document

- It should be concise
 - and at the same time should not be ambiguous.
- It should specify what the system must do
 - and not say how to do it.
- Easy to change.,
 - i.e. it should be well-structured.
- It should be consistent.
- It should be complete.

Properties of a Good SRS Document (cont...)

- It should be traceable
 - You should be able to trace which part of the specification corresponds to which part of the design, code, etc and vice versa.
- It should be verifiable
 - e.g. “system should be user friendly” is not verifiable

SRS Document (CONT.)

- SRS document, normally contains three important parts:
 - Functional requirements,
 - Non-functional requirements,
 - Goals of Implementation.

Non-functional Requirements

- **Why? - Characteristics of the system which can not be expressed as functions**
- **Nonfunctional requirements include:**
 - **Reliability issues,**
 - **Performance issues:**
 - **Example: How fast the system can produce results**
 - **so that it does not overload another system to which it supplies data, etc.**
 - **Human-computer interface issues,**
 - **Interface with other external systems,**
 - **Security, Portability, Scalability, Usability, Maintainability, etc.**

Non-Functional Requirements

- Hardware to be used,
- Operating system or DBMS to be used
- Capabilities of I/O devices
- Standards compliance
- Data representations
 - by the interfaced system

Goals of Implementation

- Goals describe things that are desirable of the system:
 - But, would not be checked for compliance.
 - For example,
 - Reusability issues
 - Functionalities to be developed in future

Goals vs. Requirements ?

Examples of Bad SRS Documents

- Unstructured Specifications:
 - Narrative essay --- one of the worst types of specification document:
 - Difficult to change,
 - Difficult to be precise,
 - Difficult to be unambiguous,
 - Scope for contradictions, etc.
- Noise:
 - Presence of text containing information irrelevant to the problem.
- Silence:
 - aspects important to proper solution of the problem are omitted.

Examples of Bad SRS Documents

- Over-specification:
 - Addressing “how to” aspects
 - For example, “Library member names should be stored in a sorted descending order”
 - Why? - Over-specification restricts the solution space for the designer.
- Contradictions:
 - Contradictions might arise
 - if the same thing described at several places in different ways.

Examples of Bad SRS Documents

- Ambiguity:
 - Literary expressions
 - Unquantifiable aspects, e.g. “good user interface”, “fantastic processing”, “ultra transfer speed”, etc.
- Forward References:
 - References to aspects of problem
 - defined only later on in the text.
 - e.g., R1.1 refers R5.2
- Wishful Thinking:
 - Descriptions of aspects
 - for which realistic solutions will be hard to find/achieve.
 - e.g., Successful Search of 1.2 billion adhaar finger-print records in 1 ns.